

FinTrack Pro

Technical Architecture Document

Full Finance Management System — Engineering Blueprint

Version: 1.0 (Draft) | Owner: Engineering | Status: For Build Kickoff
Prepared for: Backend, Frontend, DevOps

This document defines the recommended technology stack with reasoning, complete project folder structure, full database schema with relationships, and required environment configuration for FinTrack Pro v1, as scoped in the companion PRD. It is written for engineers starting implementation from zero.

1. Recommended Tech Stack & Reasoning

1.1 Frontend

Layer	Choice	Why
Framework	Next.js 14 (App Router, TypeScript)	SSR/SSG for dashboard speed, built-in API routes reduce need for separate backend for simple endpoints, strong ecosystem, easy Vercel deploy.
Styling	Tailwind CSS + shadcn/ui	Fast to build clean production-grade UI without custom CSS overhead; shadcn gives accessible pre-built components (tables, modals, forms) you can own and customize.
Charts	Recharts (bar/pie/line)	React-native charting, good defaults, easy daily/monthly/yearly data binding, lighter than D3 for this use case.
State/Data	TanStack Query (React Query)	Handles server-state caching/refetching for dashboard data cleanly, avoids manual loading/error boilerplate.
Forms	React Hook Form + Zod	Type-safe validation shared between client and API layer; catches bad finance data entry before it hits the DB.

1.2 Backend

Layer	Choice	Why
API Layer	Next.js Route Handlers (or separate NestJS if team grows)	Keeps one deployable unit for v1; NestJS is a clean migration path later if backend logic outgrows API routes.
ORM	Prisma	Type-safe DB access, auto-generated types matching schema, easy migrations — important given finance data must stay structurally strict.
Auth	NextAuth.js (Auth.js) with credentials + JWT session	Handles session/role logic without building auth from scratch; supports future SSO addition without a rewrite.
File/Export Generation	pptxgenjs (PPT), exceljs / custom dataset export (Power BI)	pptxgenjs builds real .pptx files server-side; Power BI is best served via a structured dataset/CSV or Power BI REST API push-dataset, not a native .pbix compile (see Section 4 callout).
AI Chat	Anthropic Claude API (Messages API) scoped to company dataset context	Reliable structured-output support, good for constrained Q&A; grounded strictly in uploaded finance data — reduces hallucination risk on financial figures.

1.3 Data & Infrastructure

Layer	Choice	Why
Database	PostgreSQL (via Supabase or managed RDS)	Relational integrity matters for finance records; Supabase adds instant row-level security, auth helpers, and storage in one place.
Caching	Redis (Upstash for serverless-friendly)	Cache aggregated chart queries (daily/monthly/yearly rollups) to avoid recomputing on every dashboard load.
File Storage	Supabase Storage or AWS S3	Stores generated PPT/Power BI export files and uploaded CSVs.
Hosting	Vercel (frontend + API routes) + Supabase (DB/auth/storage)	Minimal DevOps overhead for a single-company internal tool; scales fine at this size.
Background Jobs	Inngest or a simple cron (Vercel Cron)	Needed for scheduled share-value data refresh and audit-log cleanup.

■ Reasoning note: this stack assumes a lean team (1-4 engineers) and a single-tenant internal tool, matching the MVP scope from the PRD. If multi-company/SaaS (Section 8 of PRD, deferred) is greenlit later, revisit DB multi-tenancy strategy (shared schema + company_id vs. schema-per-tenant) before scaling.

2. Project File & Folder Structure

Structure below assumes Next.js 14 App Router with colocated API route handlers. This is the recommended baseline — adjust naming to team convention if one already exists.

```
fintrack-pro/
├── src/
│   ├── app/
│   │   ├── (auth)/
│   │   │   ├── login/page.tsx
│   │   │   ├── reset-password/page.tsx
│   │   │   └── (dashboard)/
│   │   │       ├── layout.tsx                # sidebar + auth guard
│   │   │       ├── dashboard/page.tsx        # bar/pie chart, daily/monthly/yearly
│   │   │       ├── employees/page.tsx        # finance dept directory
│   │   │       ├── reports/page.tsx          # PBI + PPT export, template picker
│   │   │       ├── share-value/page.tsx      # own share chart (+ comparison v2)
│   │   │       ├── performance/page.tsx      # KPI summary section
│   │   │       └── admin/
│   │   │           ├── users/page.tsx
│   │   │           └── audit-log/page.tsx
│   │   └── api/
│   │       ├── auth/[...nextauth]/route.ts
│   │       ├── finance-records/route.ts
│   │       ├── finance-records/upload/route.ts # CSV bulk upload
│   │       ├── employees/route.ts
│   │       ├── share-value/route.ts
│   │       ├── reports/generate-ppt/route.ts
│   │       ├── reports/generate-pbi/route.ts
│   │       ├── ai-chat/route.ts
│   │       └── admin/users/route.ts
│   │   └── layout.tsx
│   ├── components/
│   │   ├── charts/ (BarChart.tsx, PieChart.tsx, ShareValueChart.tsx)
│   │   ├── forms/ (FinanceEntryForm.tsx, EmployeeForm.tsx)
│   │   ├── ui/      # shadcn components
│   │   └── layout/ (Sidebar.tsx, Topbar.tsx)
│   ├── lib/
│   │   ├── prisma.ts                # Prisma client singleton
│   │   ├── auth.ts                 # NextAuth config
│   │   ├── redis.ts                 # cache client
│   │   ├── ai/claudeClient.ts       # Anthropic API wrapper
│   │   ├── export/pptGenerator.ts
│   │   ├── export/pbiDatasetBuilder.ts
│   │   └── validation/ (zod schemas per entity)
│   ├── server/
│   │   ├── services/ (financeService.ts, employeeService.ts, shareService.ts)
│   │   ├── repositories/ (financeRepo.ts, employeeRepo.ts)
│   │   └── types/ (index.d.ts)
│   ├── prisma/
│   │   ├── schema.prisma
│   │   └── migrations/
│   ├── public/
│   ├── .env.local
│   ├── .env.example
│   ├── next.config.js
│   └── tailwind.config.ts
```

```
■■■ tsconfig.json
■■■ package.json
```

Structure rationale

- **(dashboard) route group** keeps all authenticated pages under one shared layout with sidebar + role guard, without polluting the URL with a literal segment.
- **server/services + repositories split** keeps business logic (services) separate from raw DB queries (repositories) — makes it easier to unit-test finance calculations independent of the database.
- **lib/export/** isolates PPT and Power BI generation code so report-template logic doesn't leak into API route handlers.

3. Database Schema (Full, with Relationships)

PostgreSQL schema below. Each table is described in plain English after its field list. Primary keys use UUID; timestamps use **created_at/updated_at** conventions throughout for auditability, which matters for a finance system.

users

Every person who can log into the system — admins and analysts/viewers.

Field	Type	Notes
id	UUID (PK)	Primary key
email	TEXT, unique	Login identifier
password_hash	TEXT	Bcrypt/argon2 hash — never store plaintext
full_name	TEXT	Display name
role	ENUM('admin','analyst')	Controls access to admin panel and data entry
is_active	BOOLEAN, default true	Deactivation instead of hard delete
created_at / updated_at	TIMESTAMP	Audit tracking

Relationships: One user can create many finance_records and reports (see below); tracked via foreign key on those tables.

employees

The finance department staff directory — separate from the 'users' table because not every listed employee necessarily has a login (e.g., a director listed for reference only).

Field	Type	Notes
id	UUID (PK)	Primary key
full_name	TEXT	Employee name
designation	TEXT	e.g., 'Senior Financial Analyst'
department	TEXT, default 'Finance'	Kept as a field, not hardcoded, for future flexibility
email	TEXT	Contact detail
phone	TEXT	Contact detail
linked_user_id	UUID (FK → users.id), nullable	Optional link if employee also has system login
created_at / updated_at	TIMESTAMP	Audit tracking

Relationships: Optionally links to users.id (one employee may correspond to one user account, or none).

finance_records

The core financial data table — one row per reporting entry (turnover and/or profit-loss figure) tied to a specific date, which the dashboard aggregates into daily/monthly/yearly views.

Field	Type	Notes
id	UUID (PK)	Primary key
record_date	DATE	The date this figure applies to
metric_type	ENUM('turnover','profit_loss')	Distinguishes the two chart data sources
amount	NUMERIC(18,2)	Value in company's base currency
currency	TEXT, default 'INR'	Explicit currency, avoids ambiguity
notes	TEXT, nullable	Optional free-text context
created_by	UUID (FK → users.id)	Who entered this record
source	ENUM('manual','csv_upload')	Traceability for data entry method
created_at / updated_at	TIMESTAMP	Audit tracking

Relationships: Belongs to one user (created_by). Aggregated (SUM/GROUP BY date_trunc) to power both bar and pie charts across all three time granularities — no separate tables needed per time range.

share_values

Historical record of the company's own share price over time, powering the Share Value page chart.

Field	Type	Notes
id	UUID (PK)	Primary key
record_date	DATE	Date of this share price snapshot
price	NUMERIC(12,4)	Share price value
currency	TEXT, default 'INR'	Explicit currency
source	ENUM('manual','api_feed')	v1 = manual; api_feed reserved for v2 live-data integration
created_at	TIMESTAMP	Audit tracking

Relationships: Standalone table in v1 (no comparison data). v2's peer-comparison feature would add a separate 'peer_companies' + 'peer_share_values' table pair, deliberately not modeled here since it's out of v1 scope.

reports

Log of every generated Power BI dataset or PPT presentation, so users can re-download past exports without regenerating them.

Field	Type	Notes
id	UUID (PK)	Primary key
report_type	ENUM('power_bi','presentation')	Which export type
template_id	TEXT	Which of the 1-2 v1 templates was used
file_url	TEXT	Storage location (Supabase Storage/S3 path)
generated_by	UUID (FK → users.id)	Who triggered the export
date_range_start / date_range_end	DATE	The finance_records window this report covers
created_at	TIMESTAMP	Audit tracking

Relationships: Belongs to one user (generated_by); conceptually derived from finance_records over the given date range but does not duplicate that data — regenerated on demand.

ai_chat_logs

Stores AI chat question/answer pairs for auditability (important in a finance context — you want a record of what the AI told someone about company figures) and for future quality review.

Field	Type	Notes
id	UUID (PK)	Primary key
user_id	UUID (FK → users.id)	Who asked
question	TEXT	User's message
answer	TEXT	AI's response
feedback	ENUM('up','down','none'), default 'none'	Satisfaction signal referenced in PRD success metrics
created_at	TIMESTAMP	Audit tracking

Relationships: Belongs to one user (user_id).

audit_log

Generic change-tracking table for admin visibility into who changed what — referenced in the Admin Panel requirement.

Field	Type	Notes
id	UUID (PK)	Primary key

Field	Type	Notes
actor_user_id	UUID (FK → users.id)	Who made the change
action	TEXT	e.g., 'updated finance_record', 'deactivated user'
target_table	TEXT	Which table was affected
target_id	UUID	Which row was affected
metadata	JSONB, nullable	Before/after values or extra context
created_at	TIMESTAMP	Audit tracking

Relationships: Belongs to one user (actor_user_id); target_table/target_id form a loose polymorphic reference rather than a strict foreign key, since it can point at multiple tables.

3.8 Entity Relationship Summary (plain English)

- A **user** can create many **finance_records**, generate many **reports**, and send many **ai_chat_logs** messages.
- An **employee** may optionally be linked to one **user** account, but can also exist purely as a directory entry with no login.
- **finance_records** is the single source of truth for all dashboard charts and the performance KPI section — daily/monthly/yearly views are computed via date aggregation, not stored as separate rows.
- **share_values** and **reports** are independent tables that reference finance_records data conceptually (via date range) but do not hold foreign keys into it, keeping report generation decoupled from raw data storage.
- **audit_log** sits above all tables as a change-tracking layer, written to on every create/update/delete action across the app.

4. Environment Variables & Configuration Notes

4.1 Required Environment Variables

```
# Database
DATABASE_URL="postgresql://user:password@host:5432/fintrack_pro"
DIRECT_URL="postgresql://user:password@host:5432/fintrack_pro" # Prisma direct connection (migrations)

# Auth
NEXTAUTH_URL="https://your-domain.com"
NEXTAUTH_SECRET="generate-with-openssl-rand-base64-32"

# Redis (Upstash or self-hosted)
REDIS_URL="redis://default:password@host:6379"

# File Storage (choose one)
SUPABASE_URL="https://xxxx.supabase.co"
SUPABASE_SERVICE_ROLE_KEY="..."
# — or —
AWS_S3_BUCKET="fintrack-pro-reports"
AWS_ACCESS_KEY_ID="..."
AWS_SECRET_ACCESS_KEY="..."
AWS_REGION="ap-south-1"

# AI Chat
ANTHROPIC_API_KEY="sk-ant-..."

# Optional: v2 live share-price feed (not required for v1)
MARKET_DATA_API_KEY=""

# App
NODE_ENV="production"
APP_BASE_CURRENCY="INR"
```

4.2 Configuration Notes Before You Start Building

- **Never commit .env.local** — keep only .env.example in version control with placeholder values.
- **NEXTAUTH_SECRET** must be a strong random value; rotating it invalidates all active sessions, so store it in a secrets manager, not just a local file.
- **Prisma needs two connection strings** in serverless environments (Vercel + Supabase pooling):
DATABASE_URL through the pooler (e.g., PgBouncer) for runtime queries, and DIRECT_URL for running migrations.
- **Currency handling:** decide APP_BASE_CURRENCY up front — retrofitting multi-currency support into finance_records later requires a data migration.
- **Role enum is fixed at 2 values for v1** (admin, analyst) per PRD scope — do not over-engineer a permissions table yet; that's a v2 (granular RBAC) item.
- **AI chat must be prompt-scoped strictly to company data** — pass only the relevant finance_records/aggregates as context per request, never give the model unrestricted DB access, to avoid data leakage or hallucinated figures.

- **Power BI export clarification (carried over from PRD):** v1 should target either a downloadable structured dataset (CSV/Excel formatted for Power BI import) or a push-dataset via the Power BI REST API — confirm which before building `lib/export/pbiDatasetBuilder.ts`, since the two approaches differ significantly in effort.
- **Set up audit_log writes at the service layer** (`server/services/`), not scattered across API routes, so no code path can bypass logging a finance data change.

■ Accuracy note: This is an architectural recommendation based on common, well-established patterns (Next.js/Prisma/Postgres/Redis/Supabase) — no disputed external facts here. The one item worth independently verifying against your team's constraints is the Power BI integration approach (Section 4.2), since 'push-dataset API' vs. 'downloadable template' have materially different build costs and licensing implications (Power BI REST API push-datasets require a Power BI Pro/Premium workspace on the receiving end — confirm this with whoever owns the company's Power BI license before committing to that path).